

Lecture 41 : Packet Sniffing (Part 1)

Introduction

In the digital realm, proving one's identity to access resources is fundamental, a process known as authentication. For decades, passwords have served as the primary gatekeepers, a simple "something you know" that grants entry. However, as our reliance on digital systems grows, so too does the sophistication of those seeking unauthorized access. This lecture delves into the intricate world of password authentication, exploring not only how systems verify identities but also the myriad ways these defenses can be compromised. We will uncover the evolution of password storage from vulnerable plain text to more secure hashed and salted methods, and critically examine the arsenal of techniques hackers employ to crack even the most robust passwords. Understanding these mechanisms and their vulnerabilities is paramount for anyone seeking to build or secure digital systems in an increasingly interconnected and threat-laden landscape.

Core Concepts

Term	Definition	Significance
Authentication Mechanism	A method used to verify a user's identity and confirm their authorization to access a system or resource.	Essential for securing digital systems by ensuring only legitimate users gain access.
Hashing Algorithm	A one-way mathematical function that transforms input data into a fixed-size string of characters (a hash).	Crucial for secure password storage, as it prevents direct retrieval of plain text passwords even if the database is compromised.
Salt	A random string of data added to a password before it is hashed, stored alongside the hash.	Significantly enhances password security by making rainbow table attacks ineffective and forcing unique hash computations for identical passwords.

Term	Definition	Significance
Rainbow Table	A precomputed table of hashes for a vast number of potential passwords, used to quickly reverse hashes to plain text.	A powerful tool for cracking unsalted password hashes, drastically reducing the time required for attacks.
Dictionary Attack	An attempt to crack a password by systematically trying words from a predefined list (a dictionary).	Exploits the common human tendency to choose easily guessable words as passwords.
Brute Force Attack	An attempt to crack a password by trying every possible combination of characters until the correct one is found.	Guaranteed to eventually find the password but is computationally intensive, especially for long and complex passwords.
Phishing	A social engineering technique where attackers trick users into revealing sensitive information, often through fake login pages.	Bypasses technical security measures by exploiting human trust and negligence.
Social Engineering	Manipulating individuals into performing actions or divulging confidential information.	A non-technical attack vector that exploits human psychology rather than system vulnerabilities.
Offline Cracking	Cracking password hashes that have been stolen from a compromised system, without directly interacting with the live system.	Allows attackers unlimited time and resources to crack passwords without triggering system alerts or lockouts.
Strong Password	A password that is long (at least 12 characters) and incorporates a mix of uppercase, lowercase,	Minimizes the risk of successful cracking by increasing the computational

Term	Definition	Significance
	numbers, and special characters.	effort required for brute-force and dictionary attacks.

Detailed Analysis

1. Understanding Authentication Mechanisms

Authentication is the process of verifying a user's identity. While passwords are the most common, they are not the only method. Authentication mechanisms can be broadly categorized into:

- * **Something you know:** This includes traditional passwords, PINs for bank cards, or a safe word used for verification. These rely on secret information known only to the legitimate user.

- * **Something you have:** This category involves physical tokens or devices. Examples include swipe cards for secure access, one-time codes sent to a cell phone, or secure ID tokens that generate constantly changing codes. The possession of the item proves identity.

- * **Something you are:** This refers to biometric security, which uses unique biological characteristics. Examples include fingerprint scanners, retinal scans (analyzing blood vessels in the eye), and iris scans (analyzing the colored part of the eye). Biometric systems offer a high level of security as they are difficult to replicate. The lecture provides an example of a data center requiring both a swipe card ("something you have") and a fingerprint scan ("something you are") for access, highlighting multi-factor authentication.

2. The Evolution of Password Storage and Hashing

In the early days of computing, passwords were often stored in databases as plain text. When a user attempted to log in, the system would directly compare the entered password with the stored plain text.

- * **Vulnerability of Plain Text Storage:** This method was highly insecure. If a hacker gained unauthorized access to the system and downloaded the password database, they would instantly have all user passwords in clear, readable format.

- * **Introduction of Hashing:** To address this critical vulnerability, developers and system administrators introduced password hashing. A hashing algorithm acts like a one-way function:

- * It takes any input (text or binary data) and produces a fixed-length output, known as a hash or digest. For example, a 32-digit number.

- * Even a tiny change in the input data results in a completely different hash.

- * It's computationally infeasible to reverse-engineer the original input from the hash.
- * **Hashing Algorithms in Practice:** Popular hashing algorithms include MD5 (producing 128 binary digits) and SHA-2 (producing 256 bits).
- * **Secure Password Storage with Hashing:** Instead of storing plain text passwords, systems now store the hash of the password. When a user sets or changes a password, the system hashes it and stores the resulting hash. During login, the entered password is hashed using the same algorithm, and this newly generated hash is compared to the stored hash. If they match, access is granted. This means that even if a hacker steals the database, they only have a list of hashes, not the actual passwords.

3. Password Cracking Techniques (Pre-Salt)

While storing hashes was a significant security improvement, hackers quickly devised methods to crack them:

- * **Dictionary Attack on Hashes:** Hackers generate hashes from a dictionary of common words and compare these precomputed hashes against the stolen database of user hashes. If a match is found, the original password is known.
- * **Rainbow Tables:** This technique optimizes dictionary attacks. A rainbow table is a massive, precomputed list of hashes for a vast number of potential passwords. Instead of hashing words on the fly, a hacker can simply look up a stolen hash in the rainbow table to find the corresponding plain text password. This significantly speeds up the cracking process.

4. The Role of Salting in Password Security

To combat rainbow table attacks, the concept of "salting" passwords was introduced:

- * **How Salting Works:** When a new password is created, the system generates a random, unique piece of text called a "salt." This salt is prepended or appended to the user's password, and then the combined string (salt + password) is hashed. Both the salt (in plain text) and the resulting hash are stored in the database.
- * **Authentication with Salt:** During login, when a user enters their password, the system retrieves the stored salt for that user. It then combines the entered password with the retrieved salt, hashes the combination, and compares this new hash with the stored hash.
- * **Why Salt Makes Cracking Difficult:**
- * **Invalidates Rainbow Tables:** A hacker's precomputed rainbow table, which contains hashes of individual words, becomes useless. The hacker would need to combine every word in their dictionary with the specific salt for each user and then hash it, effectively forcing them to recompute hashes for every possible password-salt combination.
- * **Unique Hashes for Identical Passwords:** Even if two users choose the exact

same password, their unique salts will result in different stored hashes, preventing a single cracked hash from revealing multiple user passwords.

*** Example of Salting:**

- * User "good guy" sets password "apple".
- * System generates a random salt, e.g., `yrtzd`.
- * System combines `yrtzd` + `apple` = `yrtzdapple`.
- * `yrtzdapple` is hashed (e.g., into a 128-bit number).
- * The salt `yrtzd` and the hash are stored.
- * During login, if "good guy" types "apple", the system retrieves `yrtzd`, combines it with "apple" to form `yrtzdapple`, hashes it, and compares it to the stored hash.
- * **Hacker's Challenge with Salted Passwords:** If a hacker steals a database of salted hashes, their precomputed rainbow tables are ineffective. They must now take each stolen salt, combine it with every word in their dictionary, hash the combination, and then compare it to the corresponding user's hash. This significantly increases the computational time and resources required for cracking.

5. The Importance of Strong Passwords

Despite advancements like salting, the sheer speed of modern computing means that weak passwords remain a significant vulnerability.

- * **Cracking Speed:** A gaming PC costing under \$2000 can attempt to crack 3.2 million password hashes per second, and potentially much faster.
- * **Prevalence of Breaches:** Major services like LinkedIn and Google have experienced breaches, making it highly probable that any service we use could have its hashed passwords stolen.
- * **Characteristics of a Strong Password:** To mitigate this risk, passwords should be:
 - * At least 12 characters long.
 - * A combination of uppercase letters, lowercase letters, numbers, and special characters.

6. Advanced Password Cracking Techniques

Beyond basic dictionary and brute-force attacks, hackers employ a variety of sophisticated methods:

- * **1. Dictionary Attack:** Uses a file of common words. Even complex phrases like "admin@NPTEL2019" can be cracked quickly if they are in a dictionary or easily derivable.
- * **2. Brute Force Attack:** Tries every possible alphanumeric combination. While slow for long passwords, it's guaranteed to work eventually. It can be accelerated by using powerful hardware like GPUs or distributed computing networks.
- * **3. Rainbow Table Attack:** As discussed, these are pre-computed hash lists for

fast lookups. They are highly effective against unsalted hashes but rendered useless by salting. While salted rainbow tables exist, they are impractically large and limited in scope.

- * **4. Phishing:** An easy way to hack by tricking users. Attackers send emails with links to fake login pages that mimic legitimate services. Users unknowingly enter their credentials, which are then stolen.

- * **5. Social Engineering:** Involves manipulating people to gain information. A common tactic is for an attacker to pose as an IT support technician and directly ask for network access passwords over the phone or in person.

- * **6. Malware:** Malicious software can be installed on a victim's system to steal credentials.

- * **Keyloggers:** Record every keystroke, including passwords.

- * **Screen Scrapers:** Take screenshots during login processes.

- * **Password File Theft:** Malware can locate and copy client password files from web browsers, which, if not properly encrypted, can contain easily accessible saved passwords.

- * **7. Offline Cracking:** This occurs when a hacker obtains a password hash file from a compromised system (often via a third-party hack) and then attempts to crack the hashes on their own machines. This allows them unlimited attempts without triggering account lockouts or alerts on the target system.

- * **8. Shoulder Surfing:** A low-tech method where hackers physically observe users entering their passwords, often by posing as delivery personnel or technicians to gain access to office environments. They also look for sticky notes with passwords near computer screens.

- * **9. Spidering:** Savvy hackers analyze corporate literature, websites, sales materials, and even competitor sites to build custom wordlists relevant to the target organization. These lists are then used in dictionary or brute-force attacks, often automated by "spidering" applications similar to search engine crawlers.

- * **10. Guesses:** Exploiting human predictability. Users often choose easily memorable, non-random passwords, making them susceptible to simple guesses by attackers who understand common password patterns.

7. Practical Demonstration: Dictionary Attack with `nmap` , `crunch` , and `hydra`

The lecture demonstrates a dictionary attack against an SSH service running on a target IP address (192.168.0.51).

- * **1. Service Discovery with `nmap` :**

```
nmap 192.168.0.51
```

This command identifies open ports and running services. In the demonstration, it reveals SSH (port 22), Telnet (port 23), HTTP (port 80), and HTTPS (port 443) services, with the hostname `vyos` and OS Linux.

* **2. Dictionary Creation with `crunch` :**

```
crunch 4 4 osvy -o /root/myfile/pass.txt
```

`crunch` is used to generate a custom dictionary. Here, it creates all possible combinations of characters `o`, `s`, `v`, `y` with a minimum and maximum length of 4, storing them in `pass.txt`. This results in 256 possible passwords.

* **3. Dictionary Attack with `hydra` :**

* **Scenario 1: Unknown Username and Password:**

```
hydra -L /root/myfile/pass.txt -P /root/myfile/pass.txt 192.168.0.51 ssh
```

This command attempts to log in to the SSH service using every entry in `pass.txt` as both the username (`-L`) and the password (`-P`).

* **Scenario 2: Known Username (`vyos`):**

```
hydra -l vyos -P /root/myfile/pass.txt 192.168.0.51 ssh
```

Knowing the username (`-l vyos`) significantly reduces the search space. The attack successfully identifies `vyos` as both the username and password.

* **4. Verification:**

```
ssh vyos@192.168.0.51
```

Using the cracked credentials (`vyos` for both username and password), the attacker successfully logs into the target system.

* **Other Tools:** Other tools for dictionary attacks include `ncrack` and `medusa`.

Key Takeaways

- [] Authentication mechanisms include "something you know," "something you have," and "something you are."
- [] Early password storage in plain text was highly vulnerable to database breaches.
- [] Hashing algorithms transform passwords into fixed-length, irreversible strings, improving security by storing hashes instead of plain text.

- [] Rainbow tables are precomputed hash lists that can quickly crack unsalted password hashes.
- [] Salting involves adding a random, unique string to a password before hashing, making rainbow tables ineffective and significantly increasing cracking difficulty.
- [] Strong passwords (12+ characters, mixed case, numbers, special characters) are crucial due to the speed of modern cracking techniques.
- [] Common password cracking methods include dictionary attacks, brute-force attacks, phishing, social engineering, malware, and offline cracking.
- [] Tools like `nmap` , `crunch` , and `hydra` can be used to perform dictionary attacks against network services.

Conclusion

The journey through password authentication and cracking reveals a continuous arms race between those who secure systems and those who seek to exploit them. From the rudimentary days of plain text passwords to the sophisticated defense of salted hashes, each advancement in security has been met with innovative cracking techniques. This lecture underscores that while technological solutions like hashing and salting are vital, human factors remain a critical vulnerability. The prevalence of weak passwords, susceptibility to phishing, and the effectiveness of social engineering highlight the need for both robust technical safeguards and vigilant user education. As digital identities become increasingly central to our lives, a comprehensive understanding of these concepts is not just for security professionals but for every user and developer, ensuring that the digital gates remain secure against ever-evolving threats.